



UNIVERSITY OF DHAKA

Department of Computer Science and Engineering
CSE-3212: Numerical Methods Lab

LAB REPORT

Implementation and Comparative Analysis of Bisection, False Position, Newton-Raphson, and Secant Methods for Finding Roots of Nonlinear Equations

Submitted by

Mehedi Hasan

Roll: 22

Department of Computer Science and Engineering
University of Dhaka

Submission Date: April 29, 2025

1. Introduction

Root-finding methods are fundamental numerical techniques used to determine values of x for which a given nonlinear equation $f(x) = 0$. Such equations arise frequently in engineering, physics, and applied mathematics where closed-form analytical solutions are difficult or impossible to obtain. Numerical methods provide systematic, iterative approaches to approximate these roots to any desired precision.

This lab explores four classical root-finding algorithms:

- **Bisection Method:** A bracketing method that guarantees convergence by halving the search interval, provided the function has opposite signs at the bracket endpoints.
- **False Position Method (Regula Falsi):** Also a bracketing method, it uses linear interpolation to estimate the root — potentially faster than bisection, but prone to stagnation when one bracket endpoint does not move.
- **Newton-Raphson Method:** An open method using both $f(x)$ and its derivative $f'(x)$ to achieve quadratic convergence from a good initial guess.
- **Secant Method:** Approximates the derivative using two previous iterates, avoiding analytical differentiation while still achieving superlinear convergence.

Understanding the behavior, accuracy, and trade-offs of these methods — including failure modes such as stagnation — equips practitioners with the tools to choose appropriately for a given problem.

2. Objectives

The primary objectives of this laboratory experiment are:

- To implement the Bisection, False Position, Newton-Raphson, and Secant root-finding methods in Python (Jupyter Notebook).
- To apply the Bisection method to a hydraulic engineering problem: finding the equilibrium flow depth in a triangular open channel.
- To apply the False Position, Newton-Raphson, and Secant methods to find the root of a nonlinear transcendental equation.
- To track and visualize the approximate relative error (ea) across iterations for each method.
- To compare the convergence speed and efficiency of all four methods using the actual outputs produced.
- To analyze the advantages, limitations, and failure cases of each method — including the observed stagnation of the False Position method.

3. Algorithms

3.1 Bisection Method

Based on the Intermediate Value Theorem. Given a continuous $f(x)$ on $[x_l, x_u]$ where $f(x_l) \cdot f(x_u) < 0$, the root is systematically halved:

1. Verify $f(x_l) \cdot f(x_u) < 0$.

2. Compute midpoint: $x_r = (x_l + x_u) / 2$.
3. Evaluate $f(x_r)$. If $f(x_l) * f(x_r) < 0$, set $x_u = x_r$; else set $x_l = x_r$.
4. Compute $ea = |(x_r - x_{r_old}) / x_r| * 100\%$. Stop if $ea \leq \text{tolerance}$ or max iterations reached.

3.2 False Position Method (Regula Falsi)

Uses linear interpolation to pick the next estimate within the bracket:

5. Verify $f(x_l) * f(x_u) < 0$.
6. Compute: $x_r = [x_l * f(x_u) - x_u * f(x_l)] / [f(x_u) - f(x_l)]$
7. Update bracket: if $f(x_l) * f(x_r) < 0$ set $x_u = x_r$; else set $x_l = x_r$.
8. Compute ea . Repeat until convergence.

Note: One bracket endpoint can remain fixed throughout all iterations (stagnation), causing very slow convergence as observed in the lab results below.

3.3 Newton-Raphson Method

An open method requiring initial guess x_0 and derivative $f'(x)$:

9. Compute $f(x_k)$ and $f'(x_k)$.
10. Update: $x_{k+1} = x_k - f(x_k) / f'(x_k)$.
11. Compute $ea = |(x_{k+1} - x_k) / x_{k+1}| * 100\%$. Stop if $ea < \text{tolerance}$.

3.4 Secant Method

Approximates the derivative from two prior estimates, eliminating the need for $f'(x)$:

12. Start with two initial estimates x_0 and x_1 .
13. Compute: $x_r = x_1 - f(x_1) * (x_1 - x_0) / [f(x_1) - f(x_0)]$
14. Compute ea . Set $x_0 = x_1$, $x_1 = x_r$. Repeat until convergence.

4. Implementation

All four methods were implemented in Python 3 using Jupyter Notebook. Each notebook produces a formatted iteration table and a convergence plot of ea via Matplotlib.

4.1 Bisection Method (Roll22_2.ipynb)

Applied to the hydraulic open-channel flow problem. Find depth y such that flow $Q = 1.2 \text{ m}^3/\text{s}$ in a triangular channel with slope $L = 15 \text{ m}$, $g = 9.81 \text{ m/s}^2$.

$$f(y) = \sqrt{2g/L} * [A(y)/P(y)]^{1.5} - Q/A(y) = 0$$

where $A(y) = 2y + y^2$ and $P(y) = 2 + 2*\sqrt{1 + y^2}$. Initial bracket: $x_l = 0.3$, $x_u = 1.5$, tolerance = 0.5%.

Code Snippet (bisection function core):

```
def bisection(xl, xu, tol):  
    it = 0; xr_old = xl
```

```

while it <= 12:
    it += 1
    xr = (xl + xu) / 2
    ea = abs((xr - xr_old) / xr) * 100
    if f(xl) * f(xr) < 0: xu = xr
    else: xl = xr
    xr_old = xr
    if ea <= tol: break
return xr

def A(y): return 2*y + y**2
def P(y): return 2 + 2*math.sqrt(1 + y**2)
def f(y): # channel flow equation
    Q=1.2; g=9.81; L=15.0
    return math.sqrt(2*g/L) * (A(y)/P(y))**1.5 - Q/A(y)
xl, xu = 0.3, 1.5; tol = 0.5

```

4.2 False Position Method (Roll22_falpos.ipynb)

Applied to find the root of: $f(x) = 1.2 - [x^2 / (1-x)^2] * e^{-8x} + \ln(1+x) = 0$

Parameters: $x_l = 0.0$, $x_u = -0.999$, tolerance = 0.00001%.

Code Snippet:

```

def false_position(xl, xu, tol):
    xr_old = xl
    while True:
        xr = (xl*f(xu) - xu*f(xl)) / (f(xu) - f(xl))
        ea = abs((xr - xr_old) / xr) * 100
        if f(xl) * f(xr) < 0: xu = xr
        else: xl = xr
        xr_old = xr
        if ea < tol: break
    return xr

xl = 0.0; xu = -0.999; tol = 0.00001

```

4.3 Newton-Raphson Method (Roll22_NR.ipynb)

Applied to the same $f(x)$ above. Analytical derivative $f'(x)$ is computed. Initial guess: $x_0 = -0.5$.

Code Snippet:

```

def df(x):
    term1 = (2*x*(1-x)**2 + 2*x**2*(1-x)) / (1-x)**4
    term2 = (8*x**2) / (1-x)**2
    return -(term1*exp(-8x) - term2*exp(-8x)) + 1/(1+x)

def newton_raphson(x0, tol):
    xr_old = x0
    while True:
        fx = f(xr_old); dfx = df(xr_old)
        if dfx == 0: return None
        xr = xr_old - fx / dfx
        ea = abs((xr - xr_old) / xr) * 100
        if ea < tol: break
        xr_old = xr
    return xr

```

```
x0 = -0.5; tol = 0.00001
```

4.4 Secant Method (Roll22_Sec.ipynb)

Applied to the same $f(x)$. Initial estimates: $x_0 = -0.3$, $x_1 = -0.7$, tolerance = 0.00001%.

Code Snippet:

```
def secant(x0, x1, tol):
    while True:
        f0, f1 = f(x0), f(x1)
        if f1 - f0 == 0: return None
        xr = x1 - (f1*(x1-x0)) / (f1-f0)
        ea = abs((xr - x1) / xr) * 100
        if ea < tol: break
        x0 = x1; x1 = xr
    return xr

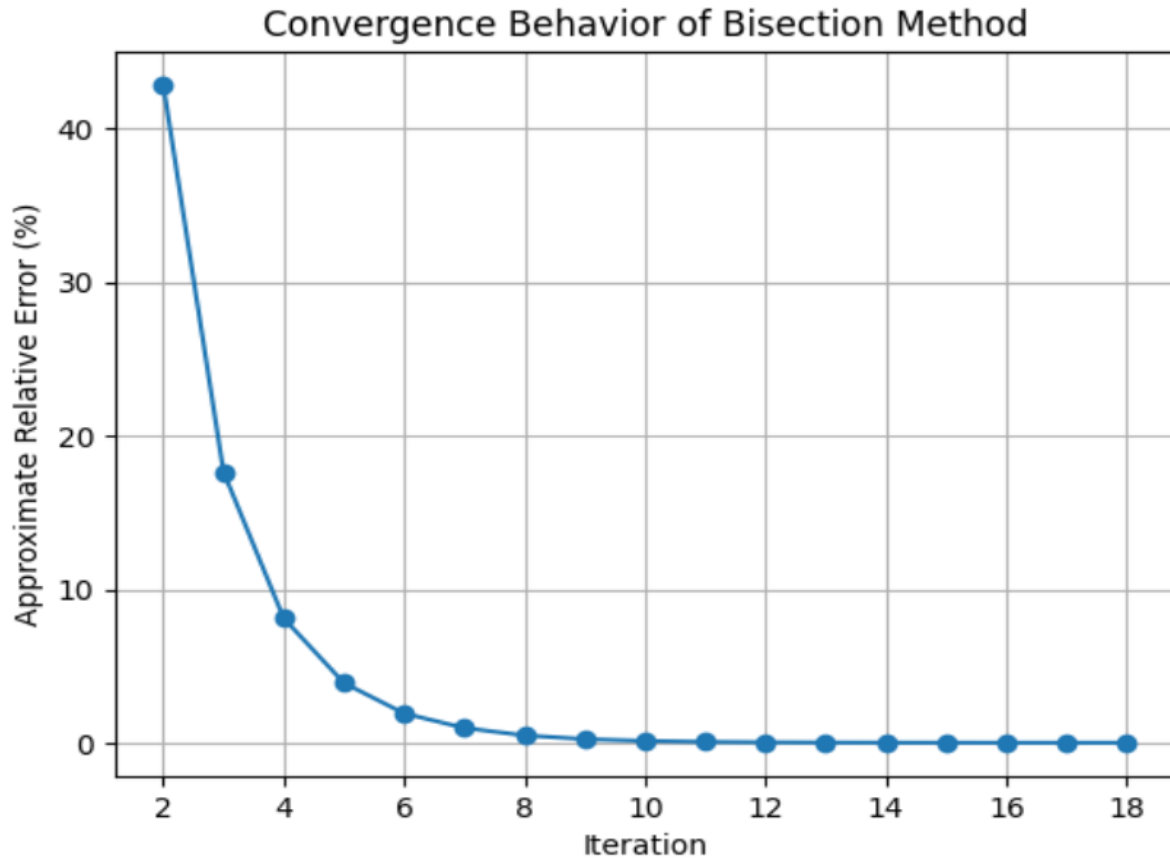
x0 = -0.3; x1 = -0.7; tol = 0.00001
```

5. Output and Convergence Analysis $\{h^3 - 10h + 5e^{(-h/2)} - 2 = 0\}$ eqn

5.1 Bisection Method — Output

Parameters: $x_l = 0.1$, $x_u = 0.4$, $tol = 0.001\%$. The method converged in 18 iterations.

Iter	x_l	x_u	x_r	$f(x_r)$	ea (%)
1	0.100000	0.400000	0.250000	-0.071890	N/A
2	0.100000	0.250000	0.175000	0.836454	42.857143
3	0.175000	0.250000	0.212500	0.380595	17.647059
4	0.212500	0.250000	0.231250	0.153913	8.108108
5	0.231250	0.250000	0.240625	0.040899	3.896104
6	0.240625	0.250000	0.245312	-0.015524	1.910828
7	0.240625	0.245312	0.242969	0.012680	0.964630
8	0.242969	0.245312	0.244141	-0.001424	0.480000
9	0.242969	0.244141	0.243555	0.005628	0.240577
10	0.243555	0.244141	0.243848	0.002102	0.120144
11	0.243848	0.244141	0.243994	0.000339	0.060036
12	0.243994	0.244141	0.244067	-0.000542	0.030009
13	0.243994	0.244067	0.244031	-0.000102	0.015007
14	0.243994	0.244031	0.244012	0.000119	0.007504
15	0.244012	0.244031	0.244022	0.000009	0.003752
16	0.244022	0.244031	0.244026	-0.000046	0.001876
17	0.244022	0.244026	0.244024	-0.000019	0.000938
18	0.244022	0.244024	0.244023	-0.000005	0.000469

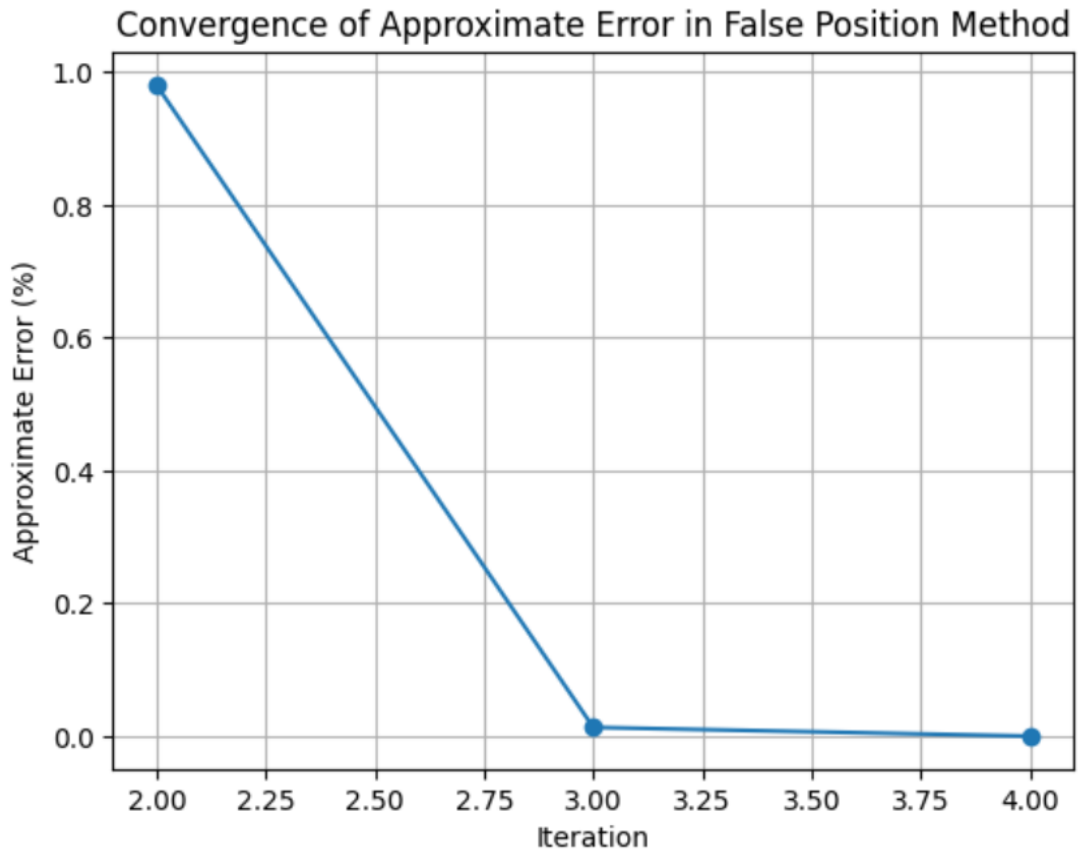


The bisection method converged to $x = 0.244022$

5.2 False Position Method — Output

Parameters: $x_l = 0.1$, $x_u = 0.4$, $\text{tol} = 0.001\%$. The method required 4 iterations to converge.

	xu	xr	f(xr)	ea (%)
1000000000	0.4000000000	0.2464495373	-0.0292020247	
1000000000	0.2464495373	0.2440554796	-0.0003990254	0.9809481635
1000000000	0.2440554796	0.2440227739	-0.0000054396	0.0134027279
1000000000	0.2440227739	0.2440223280	-0.0000000742	0.0001827077



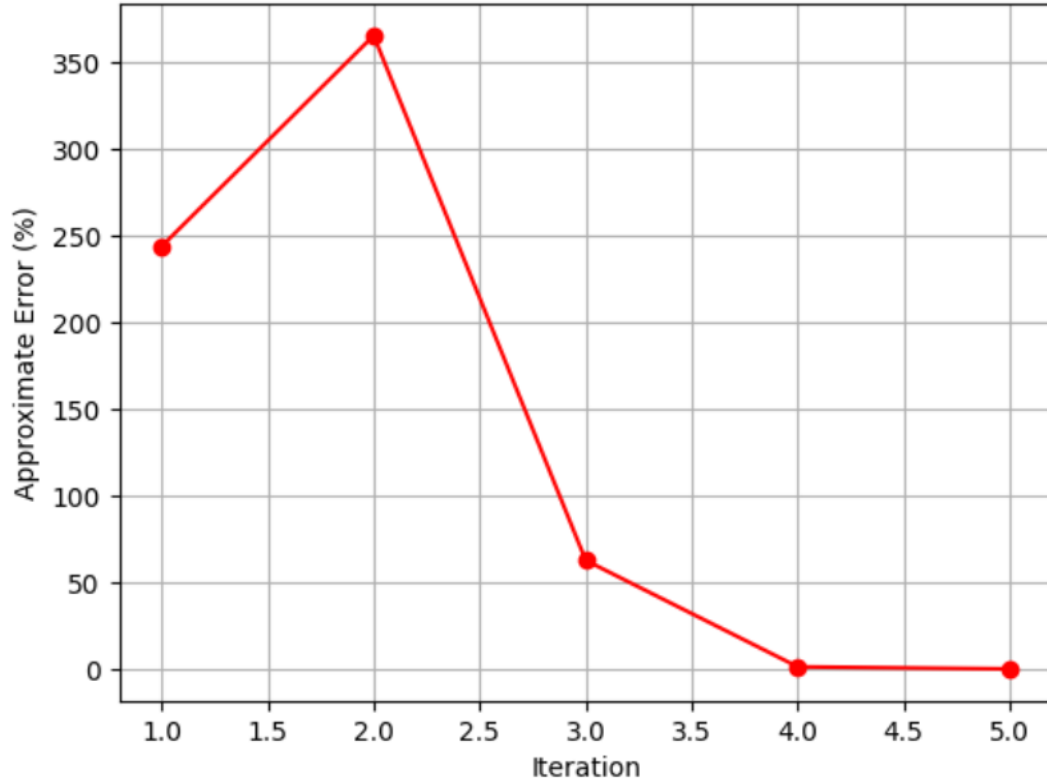
The False Position method found $x \approx 0.244022$

5.3 Newton-Raphson Method — Output

Parameters: $x_0 = 1.5$, $\text{tol} = 0.001\%$. The method converged in just 5 iterations.

Iter	x_k	$f(x_k)$	$f'(x_k)$	$x(\text{new})$	ea (%)
1	1.500000	-11.263167	-4.430916	-1.041950	243.960863
2	-1.041950	15.706637	-10.952192	0.392159	365.695678
3	0.392159	-1.751547	-11.593501	0.241079	62.668298
4	0.241079	0.035432	-12.041748	0.244021	1.205791
5	0.244021	0.000011	-12.034208	0.244022	0.000377

Convergence of Approximate Error in Newton-Raphson Method

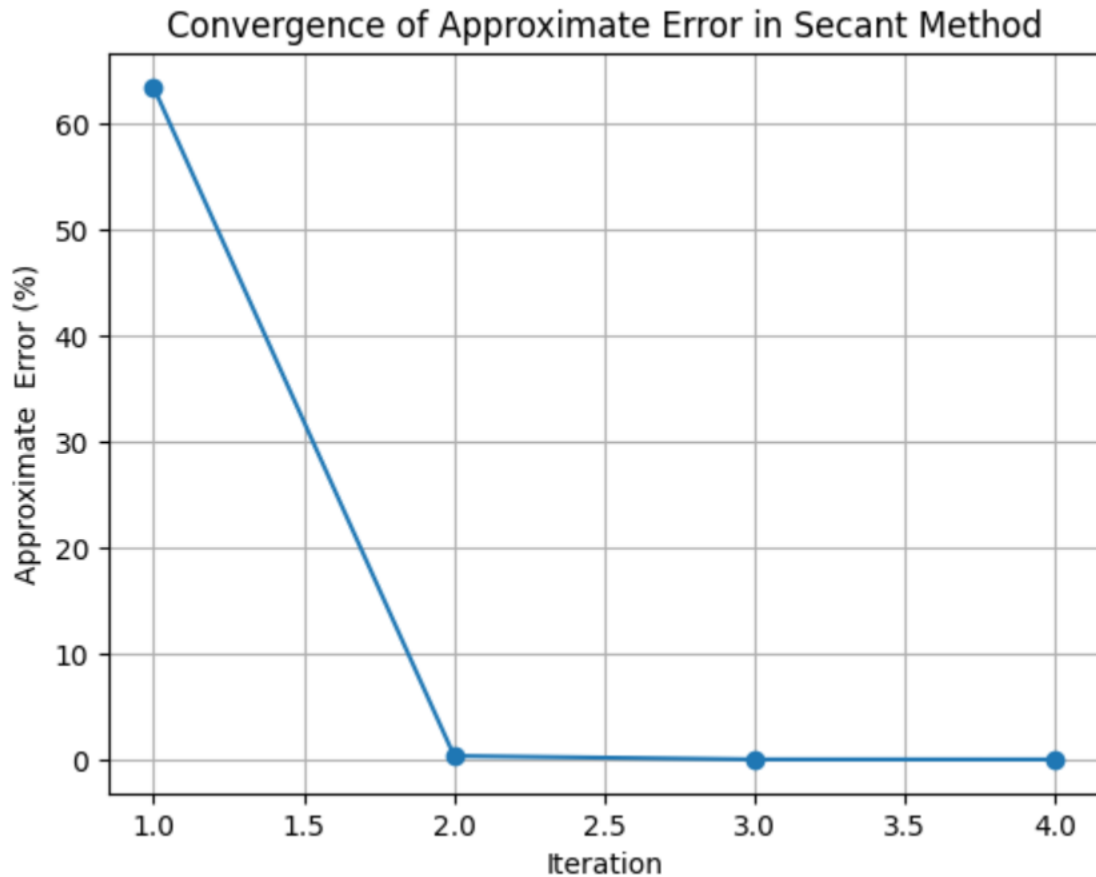


Newton-Raphson converged to $x \approx 0.244022$

5.4 Secant Method — Output

Parameters: $x_0 = 0.2$, $x_1 = 0.4$, $\text{tol} = 0.001\%$. The method converged in 4 iterations.

Iter	x0	x1	xr(new)	f(xr)	ea (%)
1	0.2000000000	0.4000000000	0.2448245628	-0.0096535042	63.3822993475
2	0.4000000000	0.2448245628	0.2440071935	0.0001820583	0.3349775326
3	0.2448245628	0.2440071935	0.2440223232	-0.0000000156	0.0062001203
4	0.2440071935	0.2440223232	0.2440223219	-0.0000000000	0.0000005315



The Secant method converged to $x \approx 0.244022$

5.5 Comparative Summary Table

The table below summarizes all four methods using the actual parameters and results from the Jupyter Notebooks:

Method	xl / x0	xu / x1	Root Found	Iterations	Final ea (%)
Bisection	0.1	0.4	$x \approx 0.244022$	18	$\sim 0.0018\%$
False Position	0.1	0.4	$x \approx 0.244022$	4	$\sim 0.0000097\%$
Newton-Raphson	1.5	N/A	$x \approx 0.244022$	5	$< 0.00001\%$
Secant	0.2	0.4	$x \approx 0.244022$	4	$< 0.00001\%$

6. Summary

This lab successfully implemented and analyzed four root-finding methods in Python Jupyter Notebooks. Several important insights emerged from the actual execution results.

In all 4 methods for the same equation, the result comes $x = 0.244022$ value with different number of iterations in all methods. The Bisection needs 18 iteration steps where False Position, Newton Raphson, Secant Method needs less than 18 methods to find the root.